

CIVL6410 Transport Network Modelling

Week 3:
Transport Network Analysis
with Graph Theory

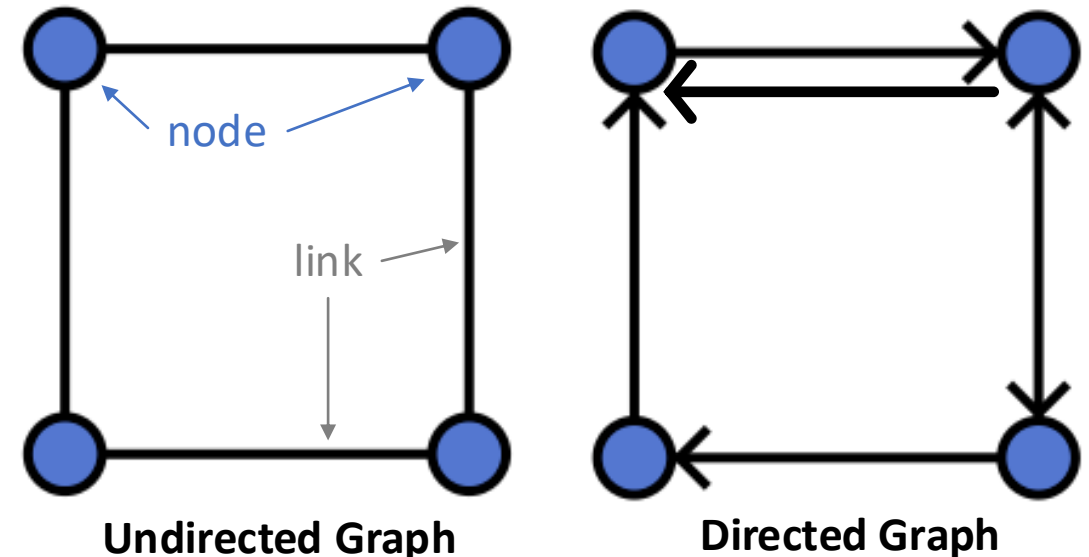
A/Prof Jiwon Kim
Semester 1, 2026

Outline

- ▶ Common Graph Theory Problems for Transportation Network Modelling
 - Shortest path problem
 - Minimum spanning tree (MST)
 - Travelling salesman problem (TSP)
 - Vehicle routing problem (VRP)
 - Maximum flow problem

Graph Theory

- ▶ **Graph theory** is a branch of **mathematics** that studies **graphs**—structures made of **nodes** (vertices) connected by **links** (edges).
 - **Nodes** represent **objects** and **links** represent **relationships** and **connections** between those objects.
- ▶ An **Undirected Graph** is a graph where links have no direction.
 - Used for **mutual relationships** (e.g., friendship networks, collaboration networks)
- ▶ A **Directed Graph** is a graph where each link has a direction, designated with an arrow.
 - Used for **one-way relationships** (e.g., follower networks, web links, traffic flow networks)



Graph Theory

- ▶ Graph theory turns **real-world systems** into **graphs** to study patterns, paths, and structures in relationships or connections between objects.

- Social Networks

- Nodes: people, Links: friendships

- Road Networks

- Nodes: intersections, Links: road segments

- Computer Networks

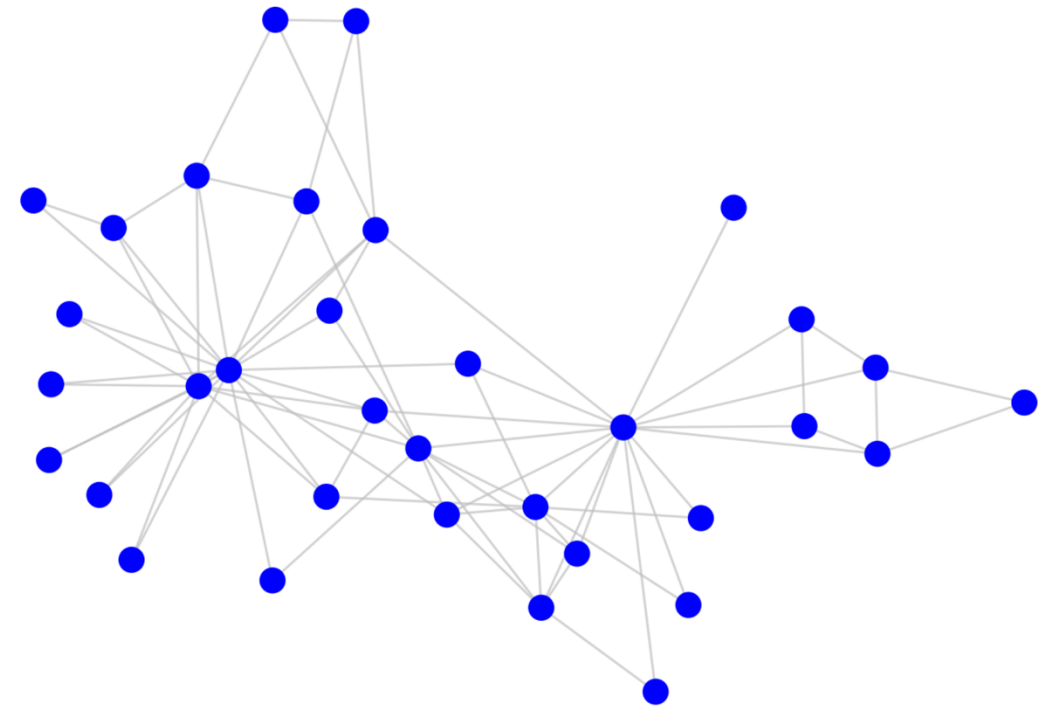
- Nodes: computers, Links: communication links

- Gene Networks (Biology)

- Nodes: genes, Links: interactions between the genes

- World Wide Web (WWW)

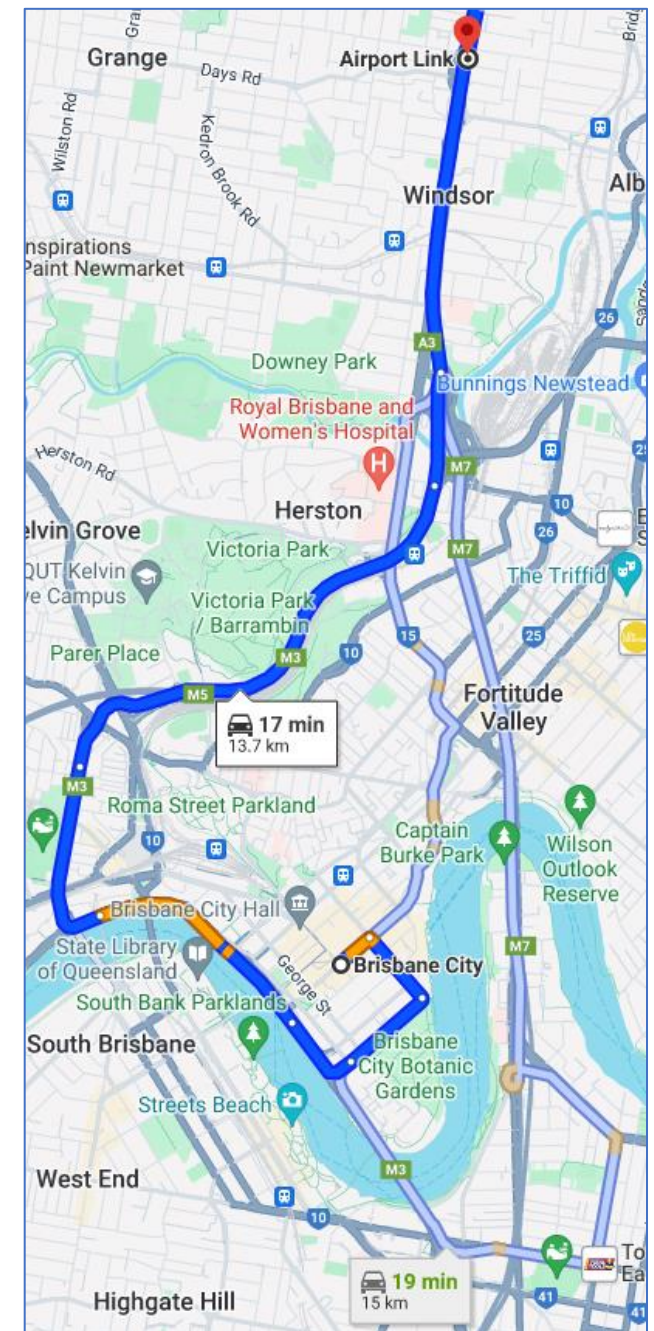
- Nodes: web pages, Links: hyperlinks between pages



Common Graph Theory Problems for Transportation Network Modelling

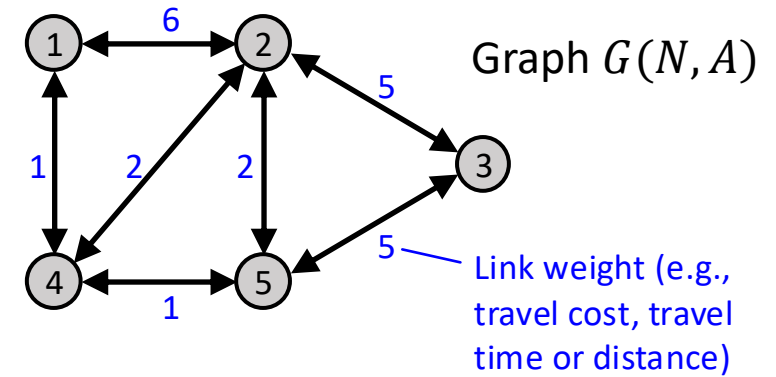
Shortest Path Problem

- ▶ “What is the **shortest path** to travel from point A to point B in a network?”
- ▶ Applications:
 - Route recommendation services (e.g., Google map)
 - Traffic assignment problems
 - The shortest path assumption for drivers’ route choice behaviour: *‘each driver wants to choose the path between their origin and their destination with the least travel time’*.



From **Brisbane City** To **Airport Link**

Shortest Path Problem



► Types of Problems:

- **Single Source** shortest path problem finds the shortest path from a single origin node to all other nodes in the graph.
- **Single Pair** shortest path problem finds the shortest path between two specific nodes in the graph.
- **All Pairs** shortest path problem finds the shortest path between all pairs of nodes in the graph.

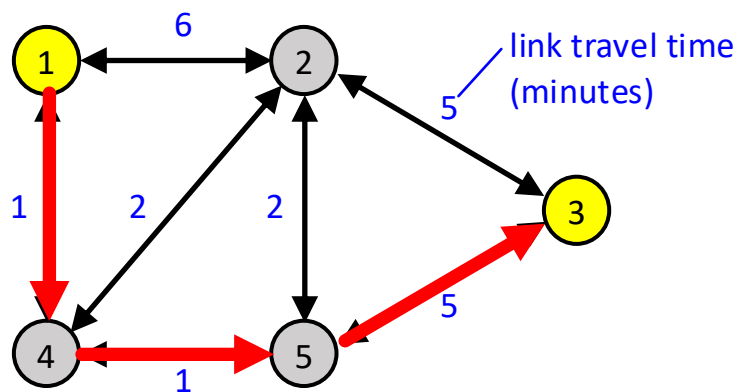
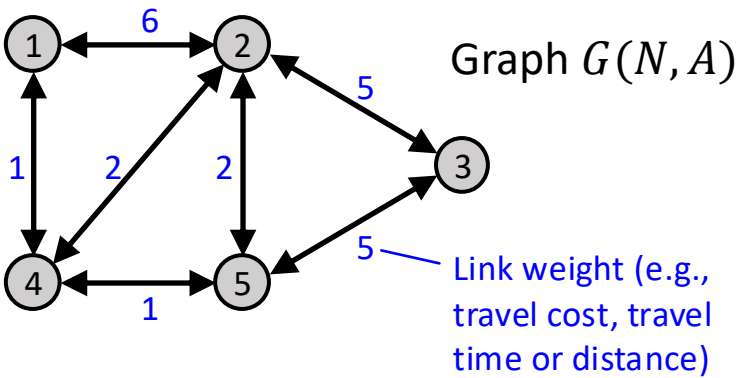
► The Presence of Negative Weights

- Traditional shortest path algorithms like **Dijkstra's Algorithm** assume link weights are positive.
- In certain scenarios, some link weights on a graph can be negative, e.g., using a certain road segment might incur savings due to time saved or fuel efficiency, resulting in a negative weight on that link.
- This introduces complexity, e.g., negative weight cycles can result in infinitely short paths.
- Algorithms like **Bellman-Ford** are specifically designed to handle negative weights.

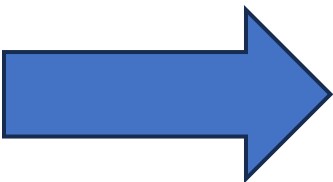
Shortest Path Problem

► In Traffic Assignment

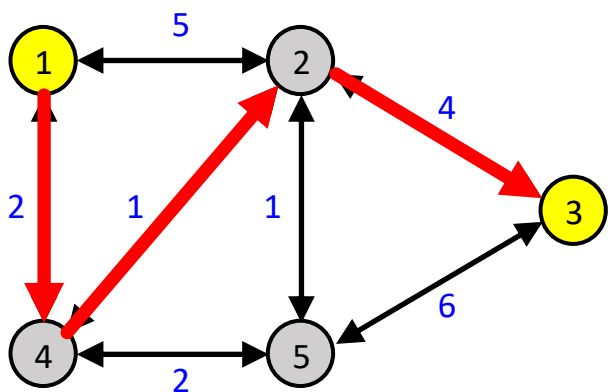
- Find the shortest path from **node 1** to **node 3**.



The shortest path: [1,4,5,3]
Path travel time: 7 minutes



Vehicles switch to the shortest path →
link travel times increase (+1) along
the shortest path, while decreasing (-1)
on the non-shortest path



The shortest path: [1,4,2,3]
Path travel time: 7 minutes

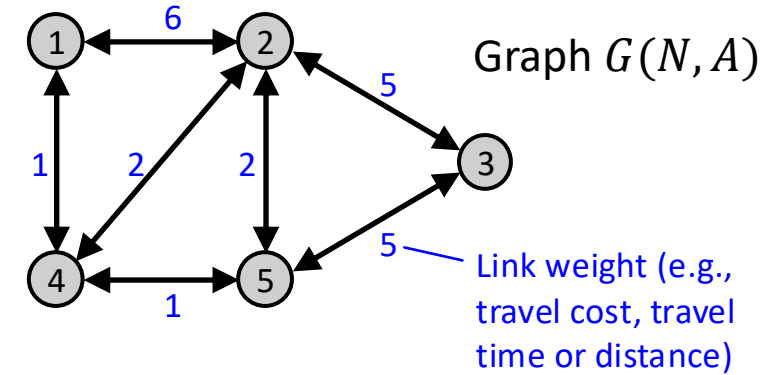
Minimum Spanning Tree (MST)

► What is a **Tree**?

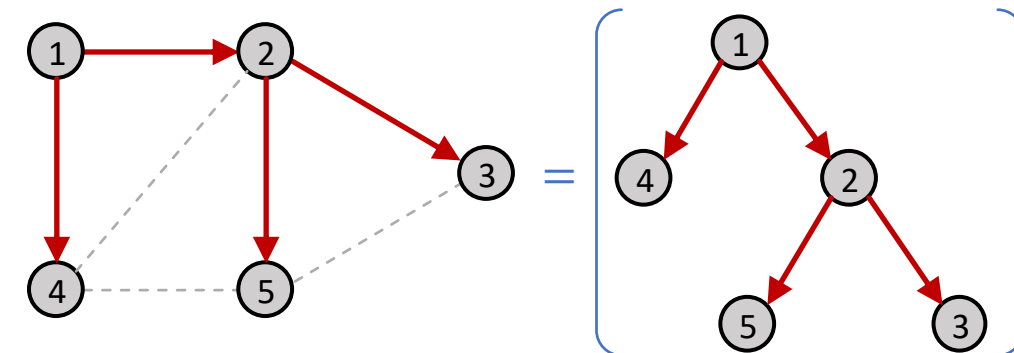
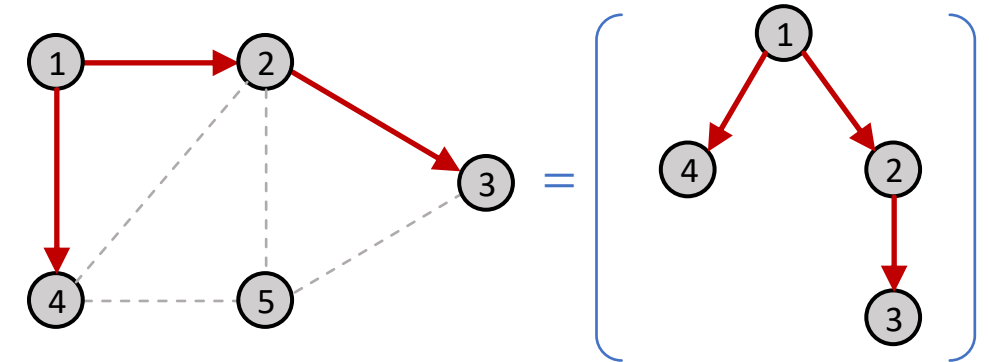
- A special type of *acyclic* network (a network without a cycle).
- Defined as a network in which there is a unique node r (called the **root**) which has the property that exactly one path exists between r and every other node in the network.

► Properties:

- A **tree** has **at least** two nodes with degree one.
- A **tree** has **at most** one path between any two nodes.
- Every node i in a tree (except the root) has a **unique** incoming link from its '**parent**' node.
- The head nodes of outgoing links of i are called '**children**' nodes.

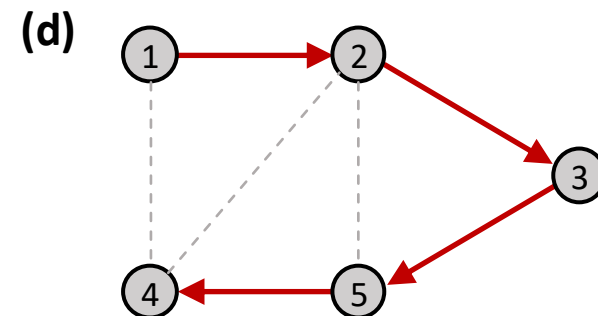
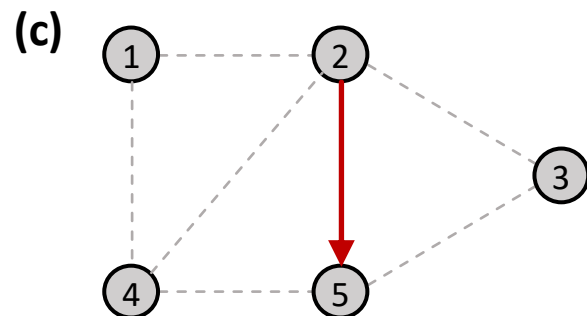
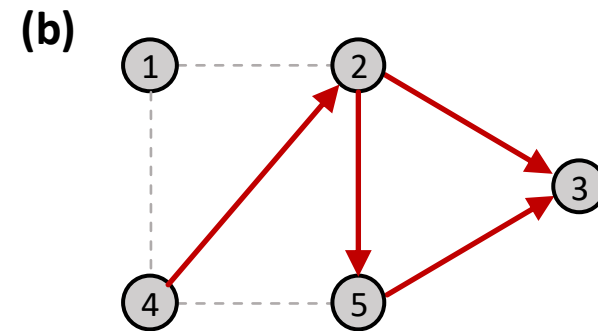
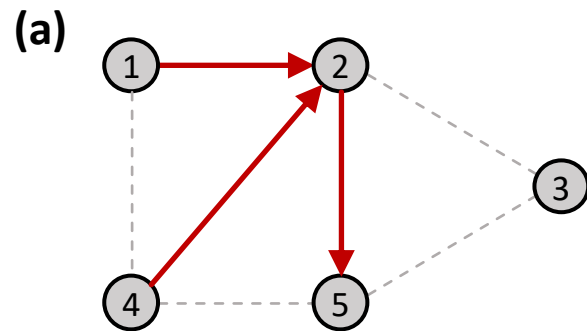


Examples of Trees



Exercise

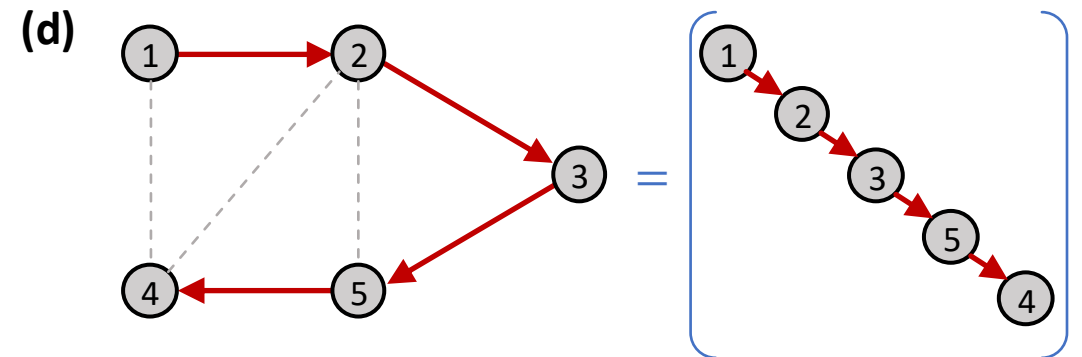
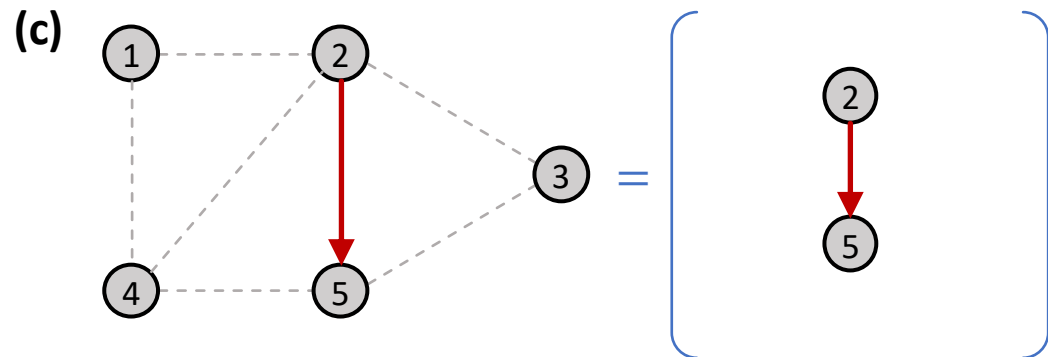
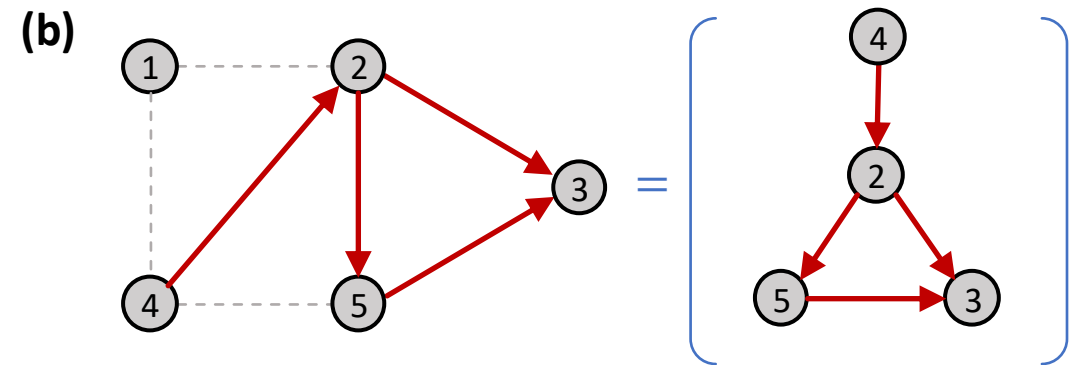
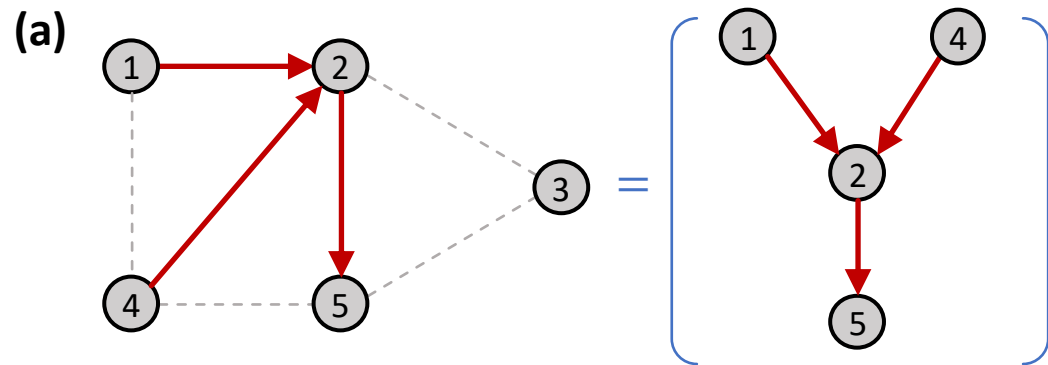
► Which of the following graphs are **trees**?



Exercise

► Which of the following graphs are **trees**?

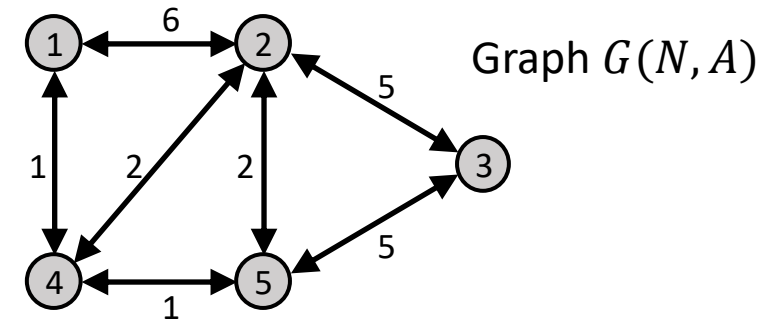
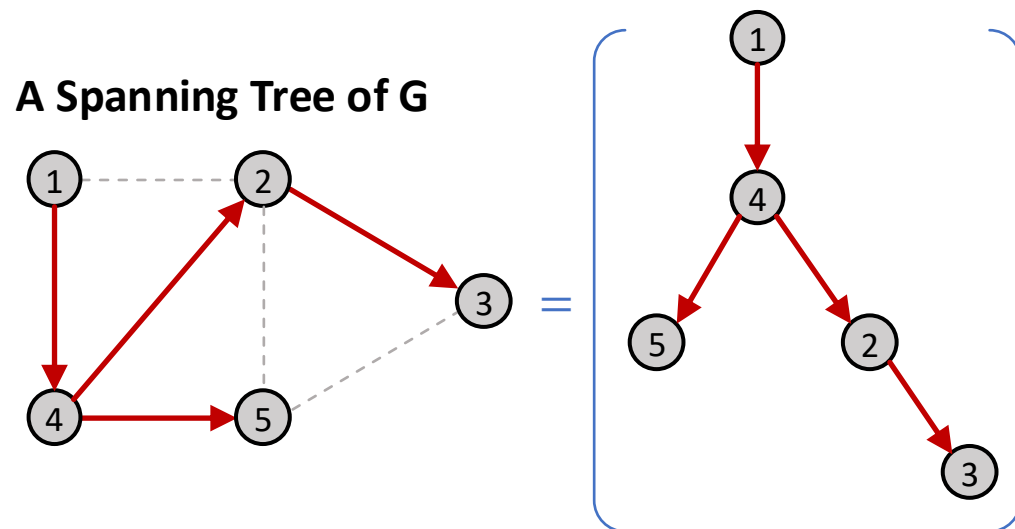
➤ (c) and (d) are trees, while (a) and (b) are not.



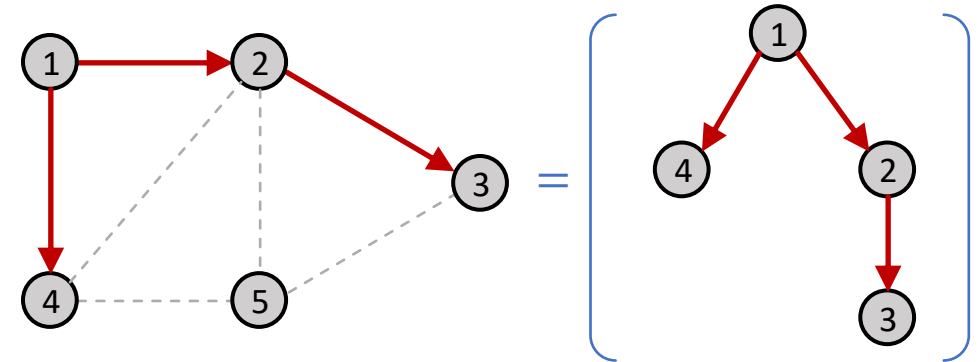
Minimum Spanning Tree (MST)

► What is a **Spanning Tree**?

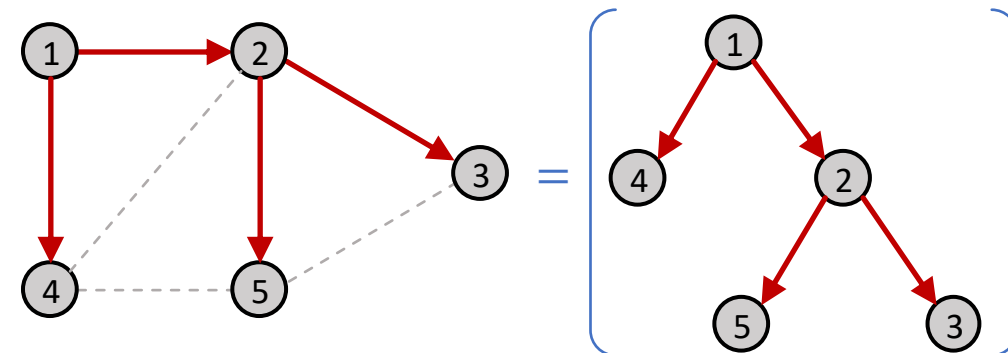
- A **spanning tree** of a graph is a **tree** that **spans the entire graph**, i.e., contains all the nodes of the original graph.
- It is a way to connect all the nodes in a graph without creating any cycles.



A Tree, but not a Spanning Tree



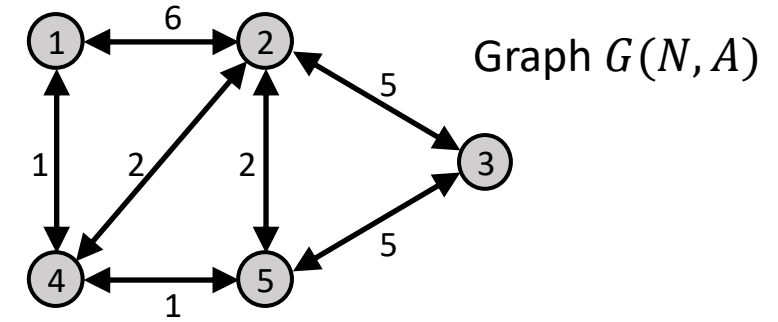
Another Spanning Tree of G



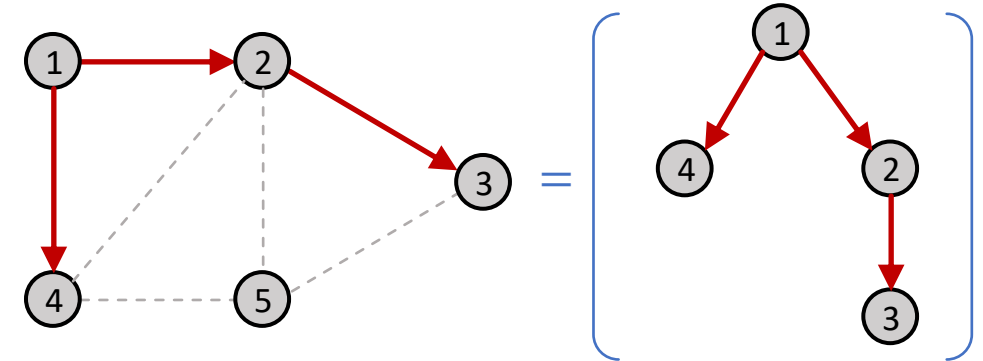
Minimum Spanning Tree (MST)

► What is a *Minimum Spanning Tree*?

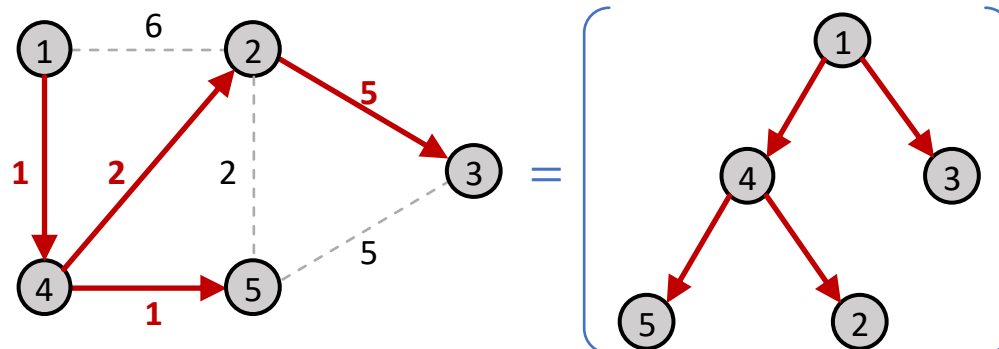
- Among all the spanning trees of a graph, the minimum spanning tree (MST) is the one with **the minimum possible sum of link weights**.
- A graph may have **multiple MSTs** that give the same total minimum weight.
- If link weight represents travel cost (travel time), it's the spanning tree with the least total cost or minimum travel time.



A Tree, but not a Spanning Tree

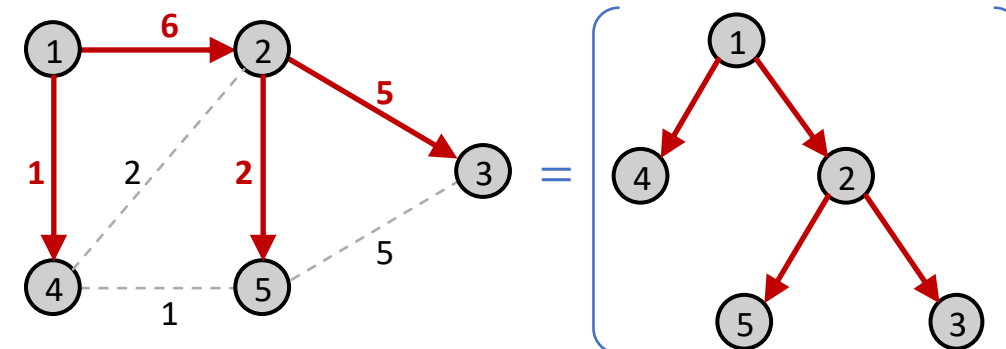


A Minimum Spanning Tree of G



Total cost = 9 (1+1+2+5)

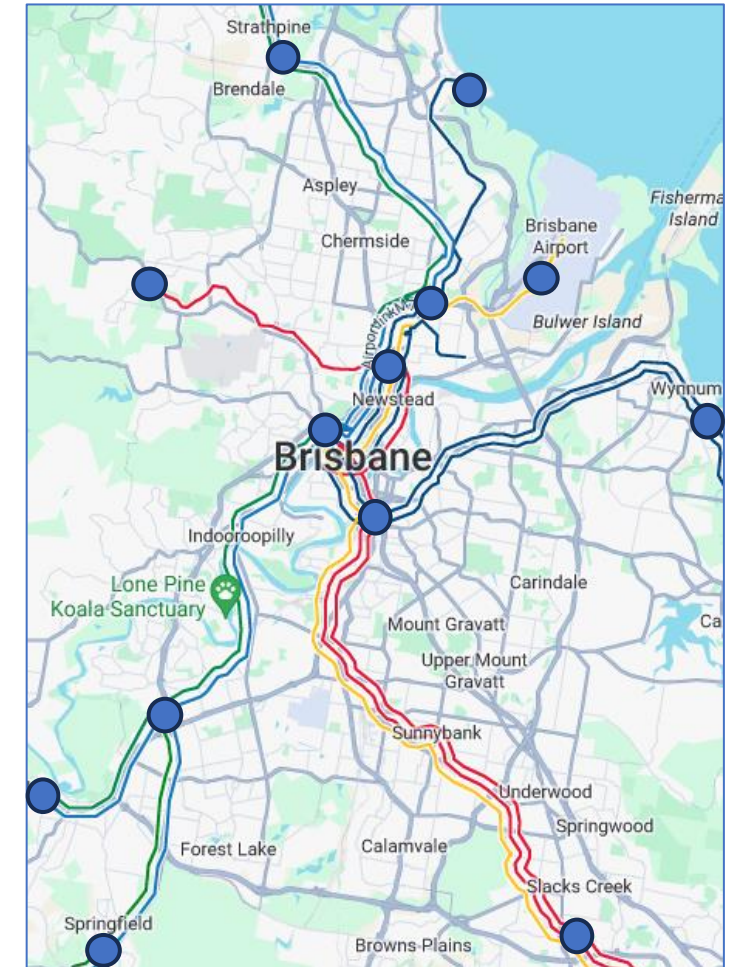
A Spanning Tree, but not an MST



Total cost = 14 (1+6+2+5)

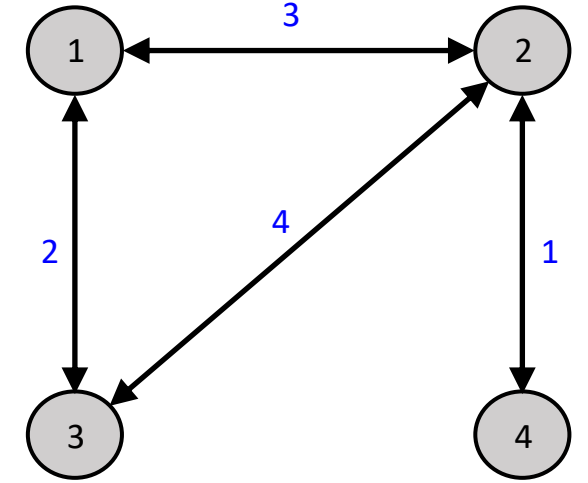
Minimum Spanning Tree (MST)

- ▶ **Spanning trees** can be used to identify possible network designs that **connect all major regions in a city**.
 - It ensures everyone can travel from their region to any other region.
 - It can guide urban planners in determining where to build new roads or design public transport networks to improve connectivity and accessibility.
- ▶ **Minimum spanning trees** can be used to select a network design that will **make the network fully connected at the lowest cost**.
 - It helps identify the most cost-effective network of routes that minimise travel time and maximise coverage.
 - It ensures optimal resource allocation under budget constraints.



Exercise

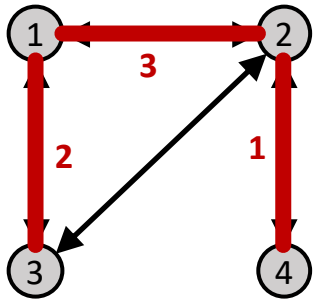
- What is the minimum cost (total link weight) covered by a minimum spanning tree in this graph?



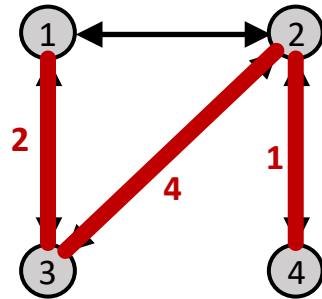
Link	Weight
(1,2) & (2,1)	3
(1,3) & (3,1)	2
(2,3) & (3,2)	4
(2,4) & (4,2)	1

Exercise

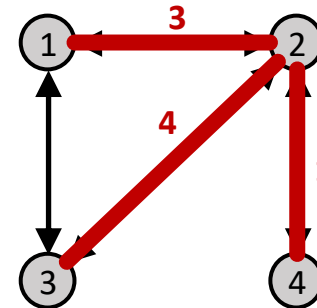
- What is the minimum cost (total link weight) covered by a minimum spanning tree in this graph?



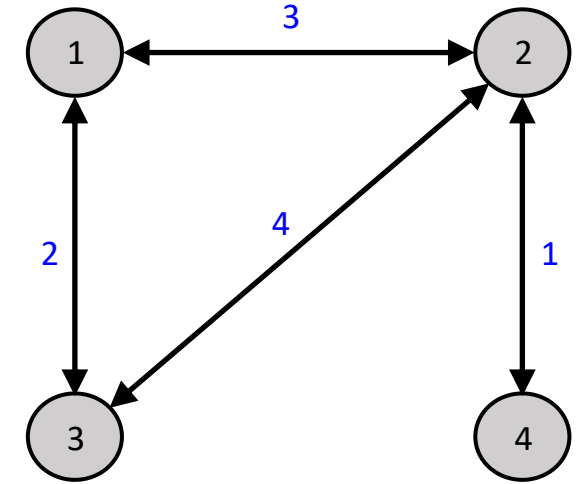
Cost = 6 (2+3+1)
Minimum Cost



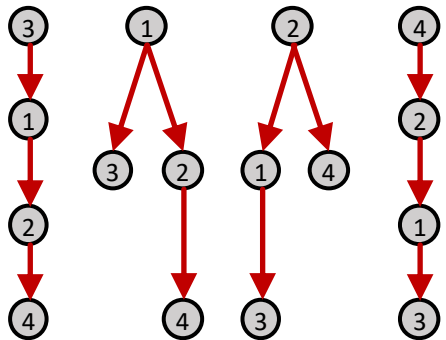
Cost = 7 (2+4+1)



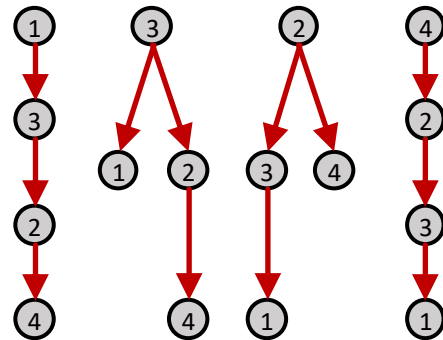
Cost = 8 (3+4+1)



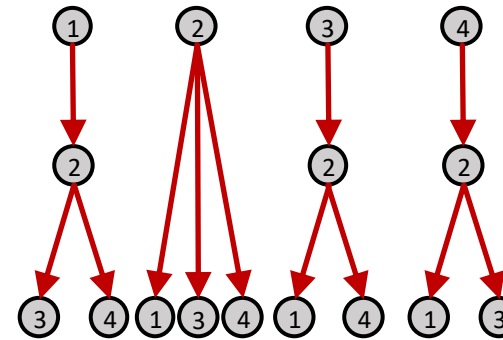
Link	Weight
(1,2) & (2,1)	3
(1,3) & (3,1)	2
(2,3) & (3,2)	4
(2,4) & (4,2)	1



Minimum Spanning Trees

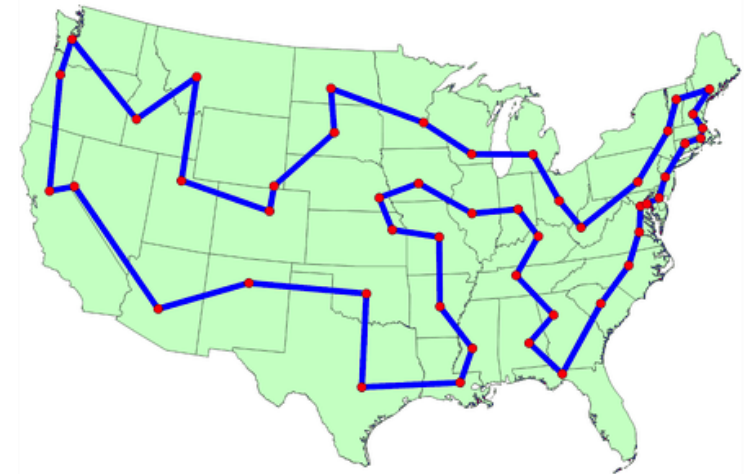


Spanning Trees



Travelling Salesman Problem (TSP)

- ▶ “Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?”
- ▶ The TSP is one of the most intensely studied problems in optimisation and computational mathematics.
- ▶ Applications in transport and logistics
 - **Vehicle routing** for public transport services, parcel delivery, and waste collection
 - **Supply chain management:** optimising supply chain logistics by finding the most efficient route between warehouses, distribution centres, and retail stores
 - **Flight planning:** optimising flight routes for aircraft, crew, and cargo

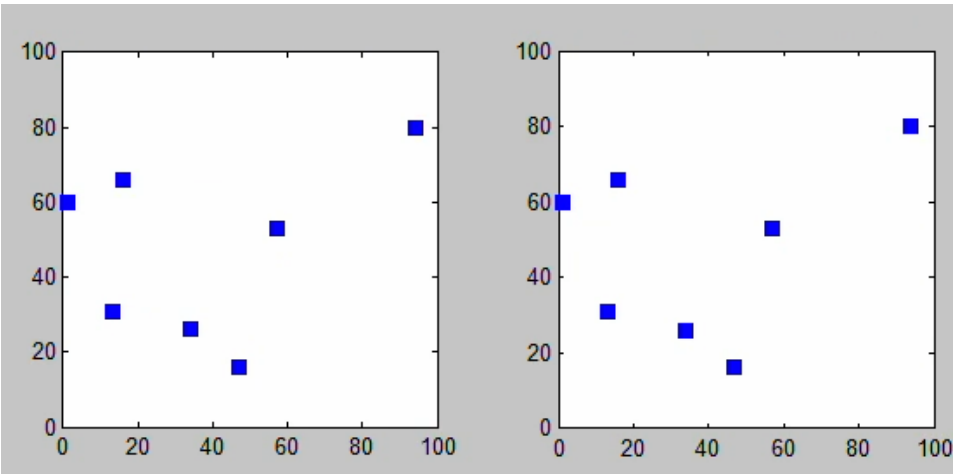


Solution to 48 States Traveling Salesman Problem
https://optimization.cbe.cornell.edu/index.php?title=Traveling_salesman_problem

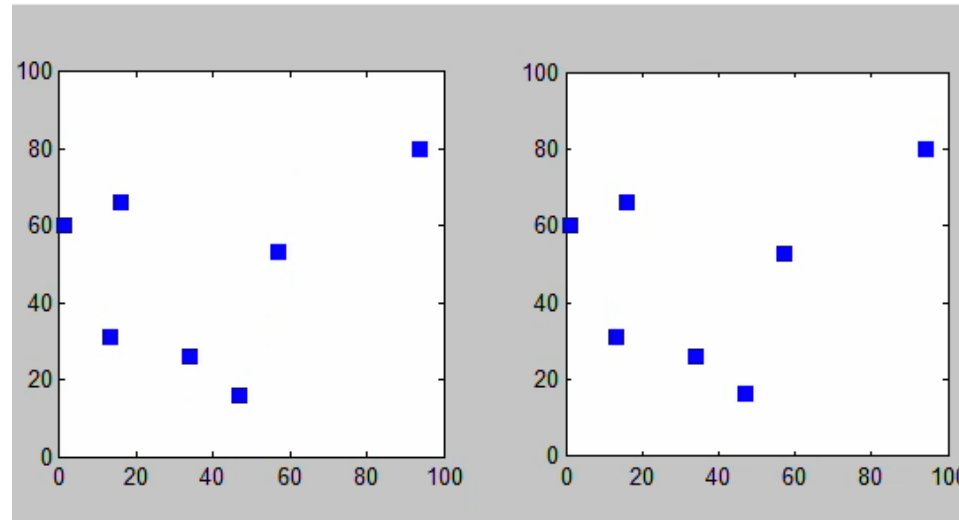
Travelling Salesman Problem (TSP)

- ▶ The TSP's computational complexity increases exponentially with the number of cities and finding the optimal solution becomes exceedingly difficult the problem grows.
- ▶ **Exact algorithms vs Heuristics**

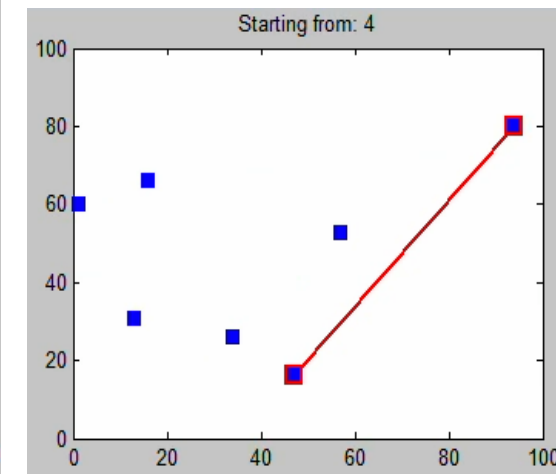
Various algorithms to solve a TSP with 7 cities (source: Wikipedia https://en.wikipedia.org/wiki/Travelling_salesman_problem)



Exact solution using the Brute Force search (360 iterations)



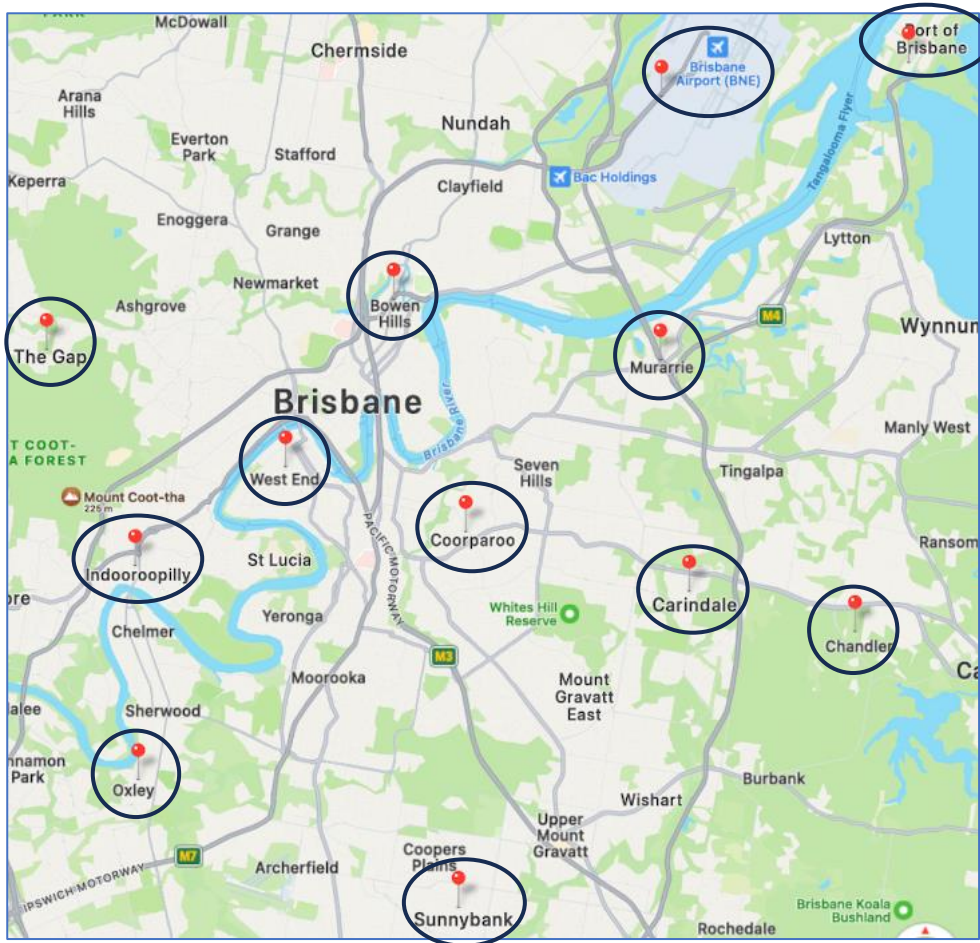
Exact solution using a Branch-and-Bound algorithm (129 iterations)



Heuristic solution using the Nearest Neighbour algorithm

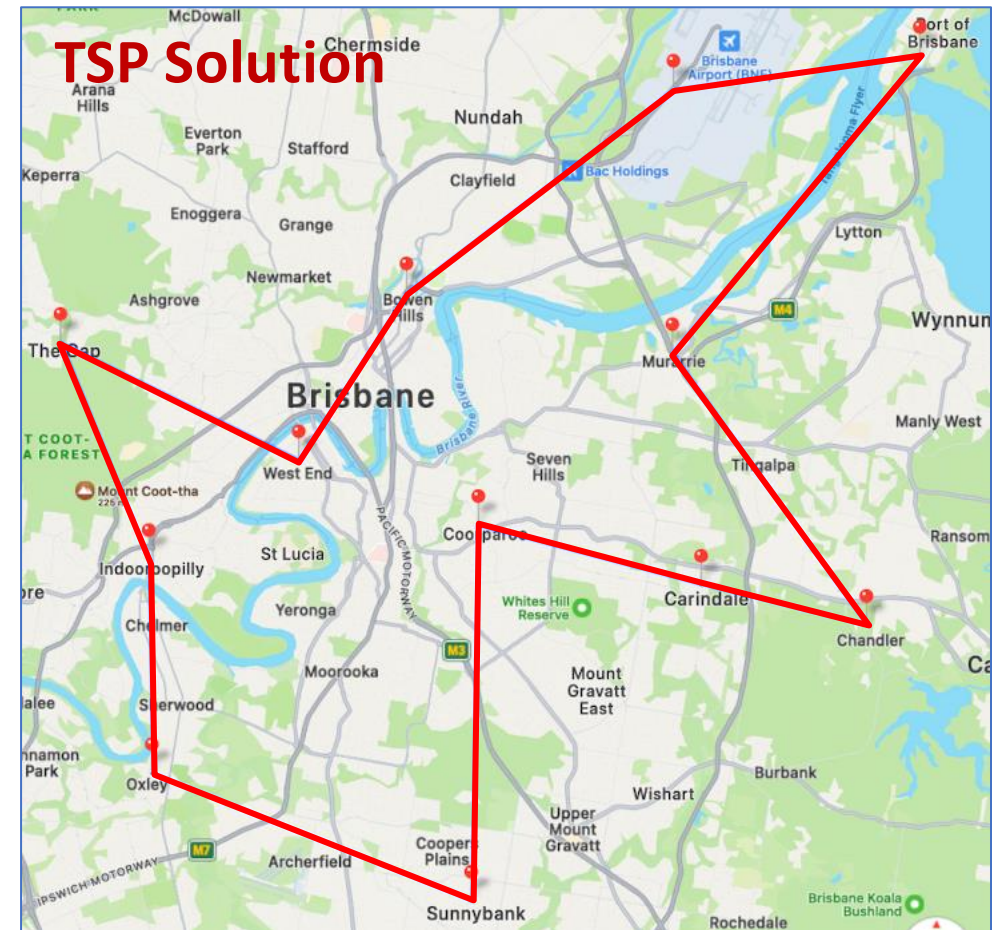
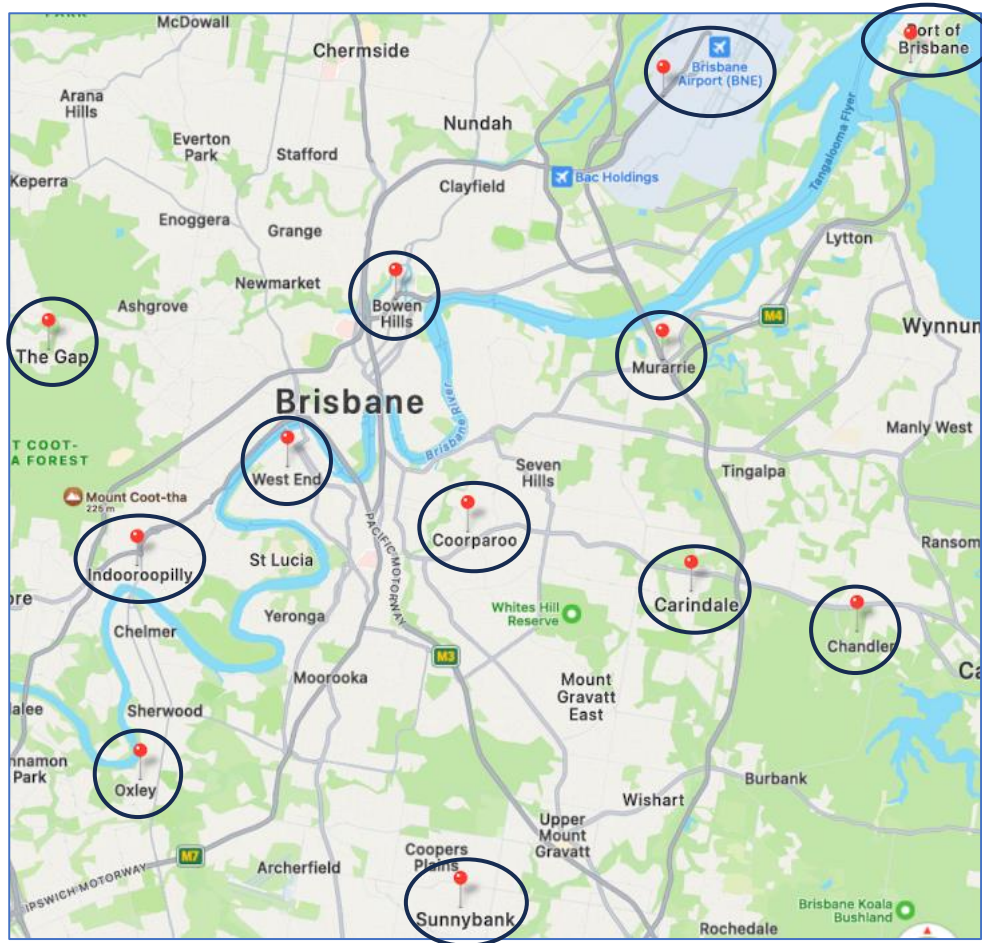
Exercise: Solve a TSP with 12 regions in Brisbane.

- Use an air distance (flying distance) for distance estimation.



Exercise: Solve a TSP with 12 regions in Brisbane.

- Use an air distance (flying distance) for distance estimation.



A TSP in a Real-World Logistics Scenario

“Amazon Last Mile Routing Challenge”



- ▶ In January 2021, Amazon launched a research competition to address the complex logistics of delivering packages efficiently in the "last mile" of delivery, which refers to the final stage of **the delivery process from a distribution centre to the end customer's location**.



<https://www.aboutamazon.com/news/transportation/amazons-electric-delivery-vehicles-from-rivian-roll-out-across-the-u-s>

- ▶ The goal was to develop an algorithm to quickly compute efficient tours by **taking into account real-world factors** such as parking availability, ease of navigation, the practicality of serving multiple locations together, and insights from experienced drivers.
- ▶ The competition drew 2,285 participants from around the globe (\$100k prize money for the winning team).

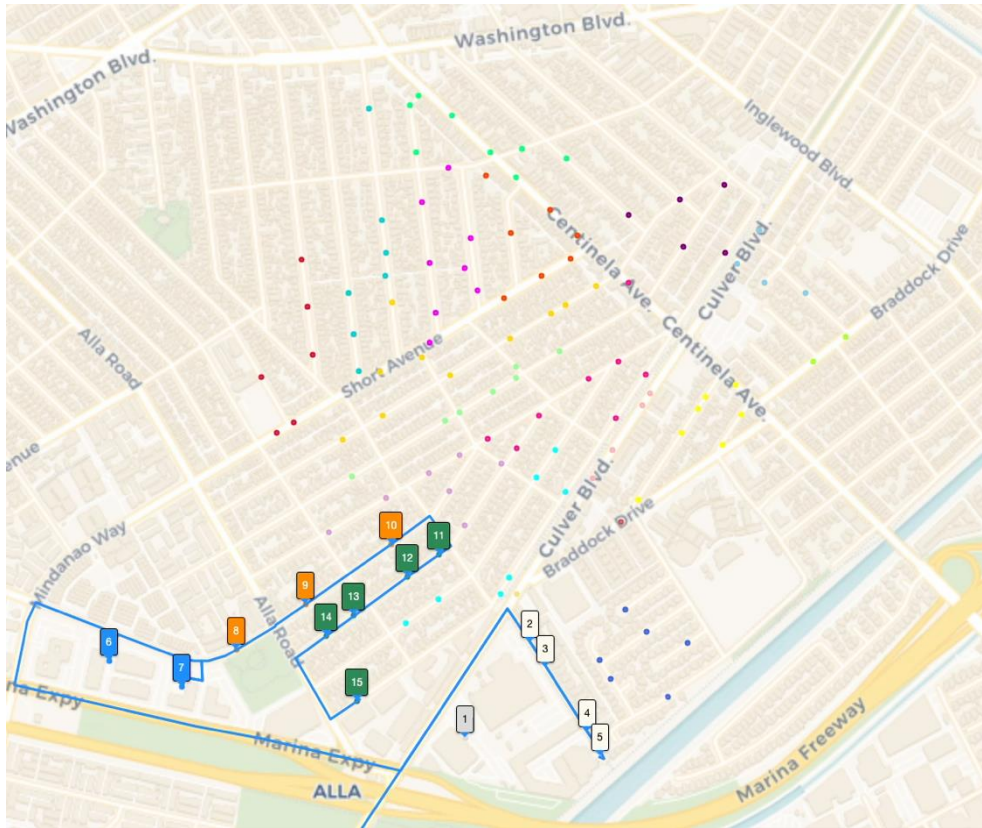


<https://routingchallenge.mit.edu/>

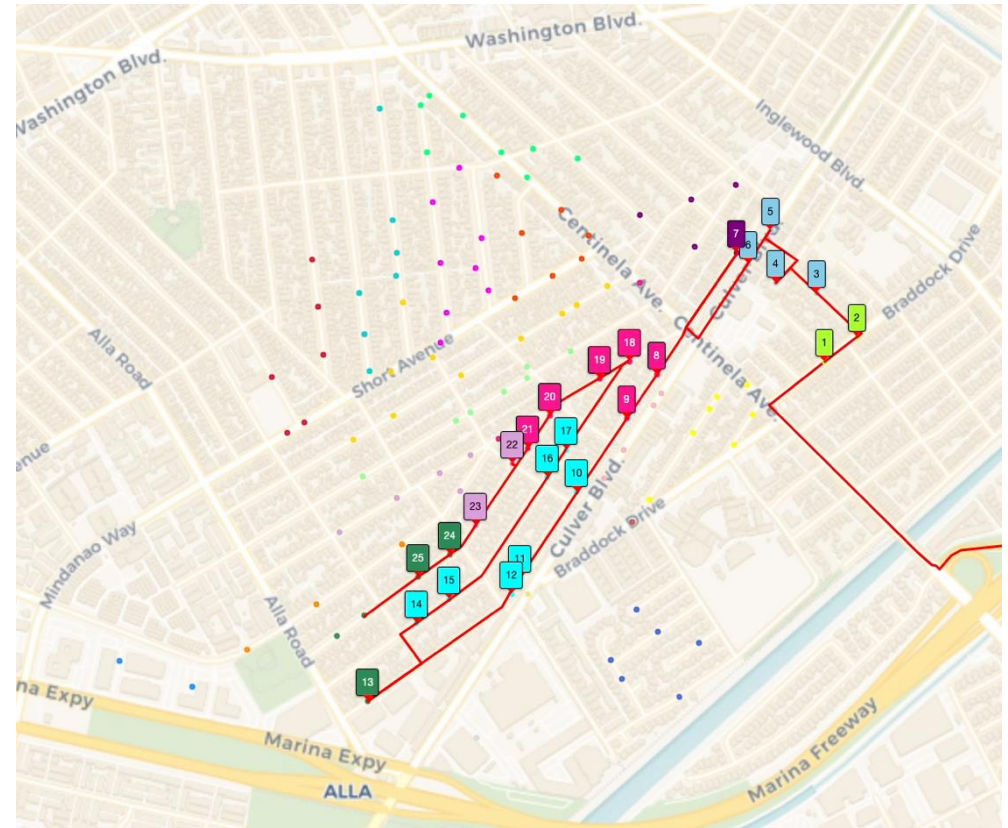
A TSP in a Real-World Logistics Scenario

“Amazon Last Mile Routing Challenge”

- Driver tours $\neq$ Shortest tours



Amazon driver tour

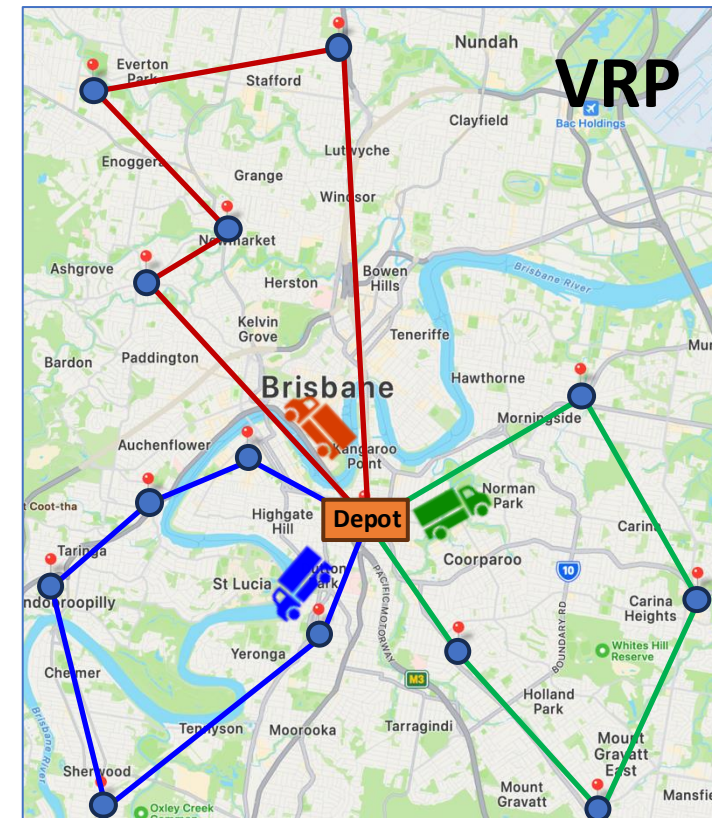
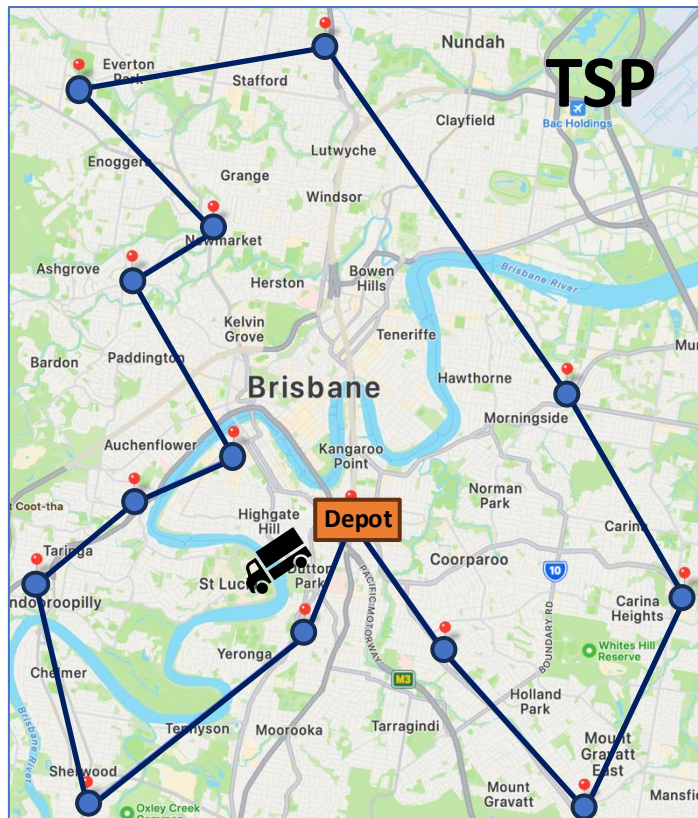


Optimal tour (the shortest tour solution of the TSP)

Animations are captured from Prof Willam Cook's homepage: <https://www.math.uwaterloo.ca/tsp/amz/maps.html>

Vehicle Routing Problem (VRP)

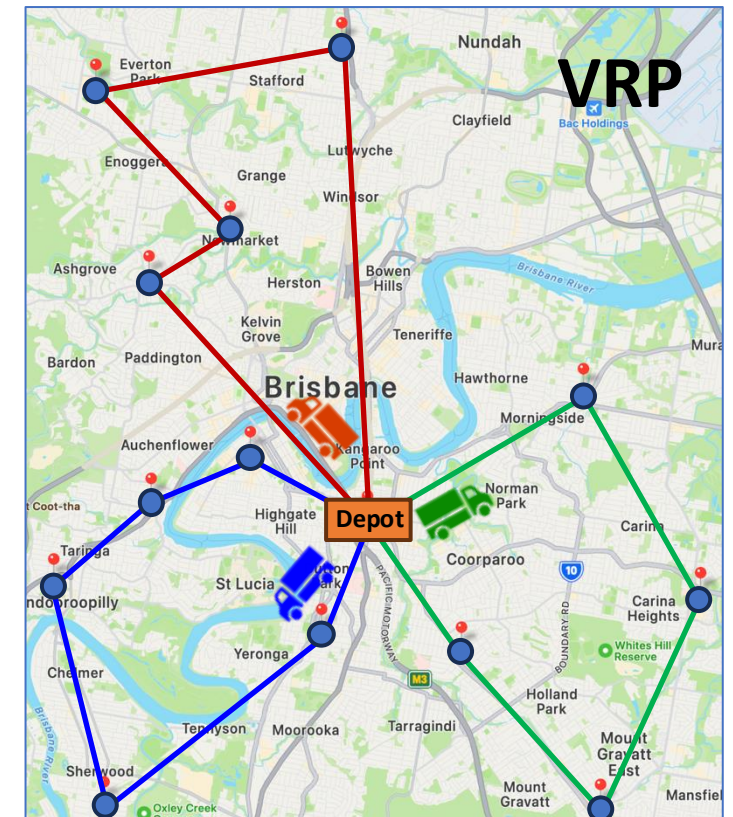
- ▶ “What is the optimal set of routes for a fleet of vehicles to traverse in order to deliver a given set of customers?”
- ▶ Vehicle Routing Problem (VRP) is a generalisation of Travelling Salesman Problem (TSP).



Vehicle Routing Problem (VRP)

► The Elements of VRP

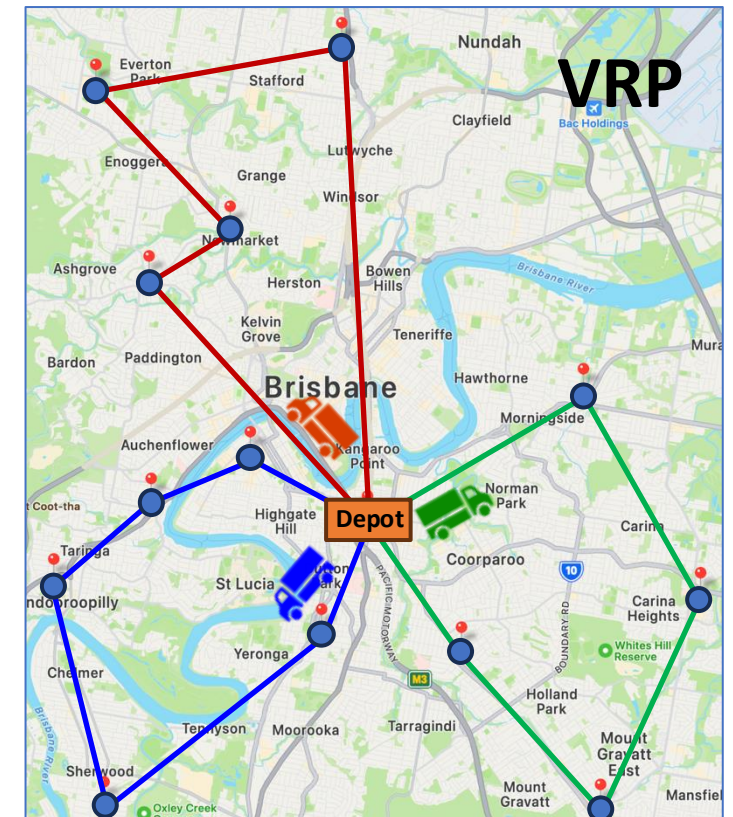
- **Depot:** The central point from which all vehicles start and return after completing their routes. It's where vehicles are loaded with goods or services before heading out to serve customers.
- **Customers:** The destinations that need to be serviced. Each customer has specific demands (such as quantities of goods to be delivered or services required) and may have time windows within which they must be served.
- **Vehicles:** The fleet of vehicles available for routing. Each vehicle has a capacity limit, meaning it can only carry a certain amount of goods or provide a certain level of service.
- **Routes:** The goal is to determine the optimal routes for each vehicle, considering factors such as distance travelled, time taken, vehicle capacity constraints, and any other relevant costs (e.g., fuel costs, labour costs).



Vehicle Routing Problem (VRP)

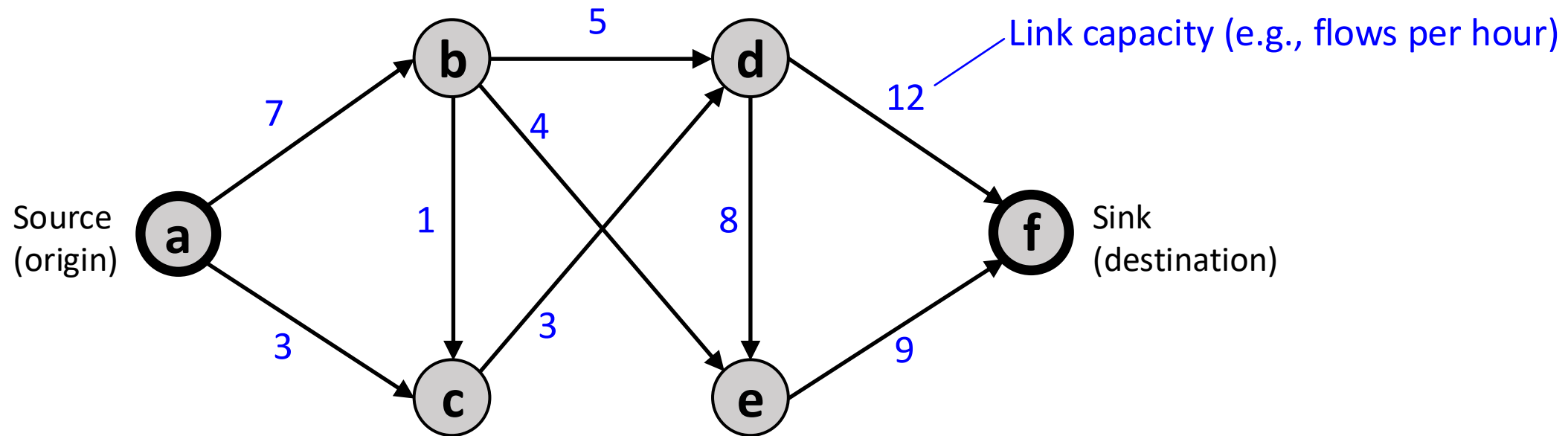
► The Objectives of VRP

- **Total distance travelled:** Minimise the overall distance travelled by all vehicles.
 - **Total time taken:** Minimise the time taken for all vehicles to complete their routes.
 - **Number of vehicles used:** Minimise the number of vehicles required to service all customers.
 - **Costs:** Minimise costs associated with vehicle operation, including fuel, labour, and vehicle wear and tear.
- Real-world VRP scenarios often involve **multiple constraints** (vehicle capacity limits, time windows for customer visits, vehicle start and end at depot) and **multiple, potentially conflicting objectives** (minimising total distance travelled vs. minimising the number of vehicles used), which is a complex optimisation challenge.



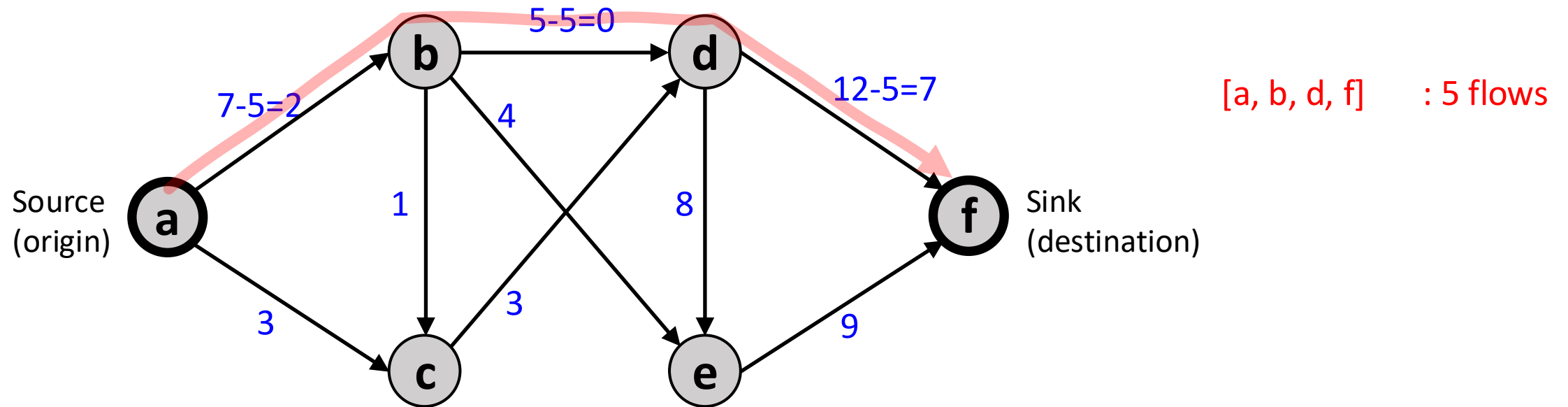
Maximum Flow Problem

- “With an infinite input source, how much “flow” can we push through the network (from a **source** (origin) to a **sink** (destination) given that each link has a certain **capacity**?”



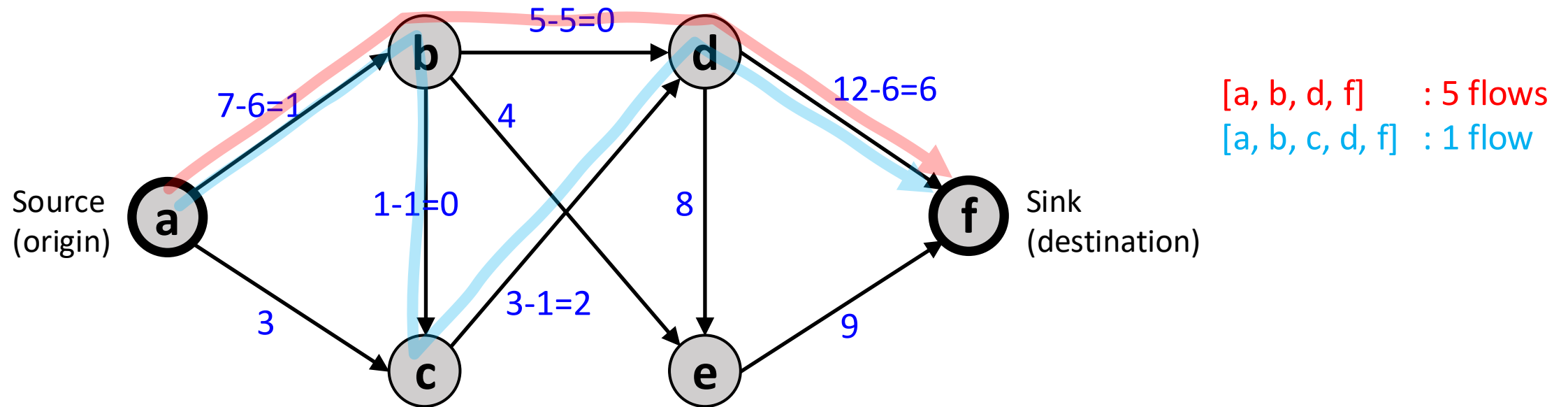
Maximum Flow Problem

- “With an infinite input source, how much “flow” can we push through the network (from a **source** (origin) to a **sink** (destination) given that each link has a certain **capacity**?”



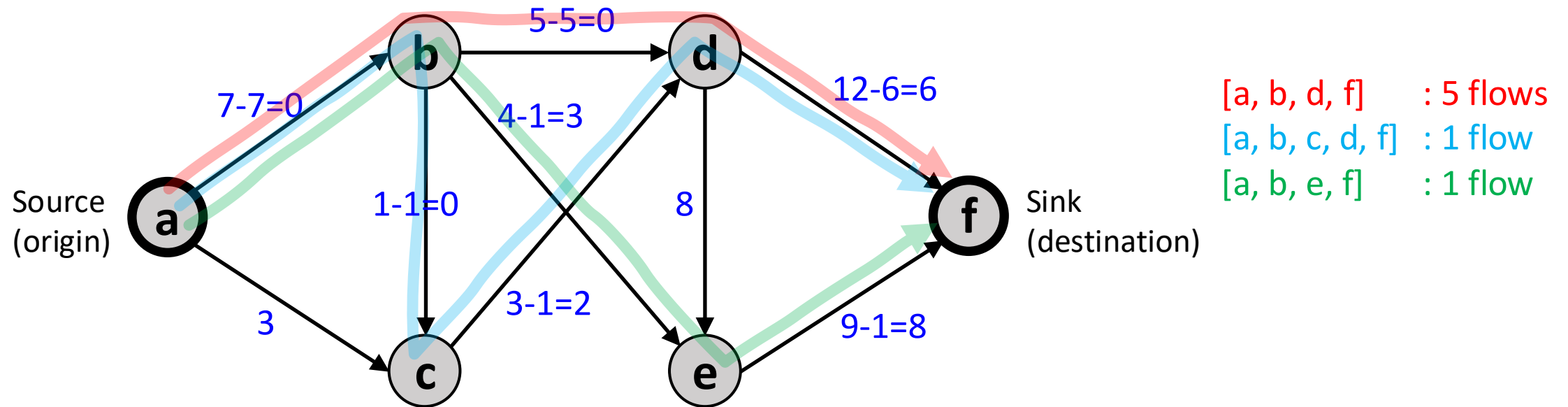
Maximum Flow Problem

- “With an infinite input source, how much “flow” can we push through the network (from a **source** (origin) to a **sink** (destination) given that each link has a certain **capacity**?”



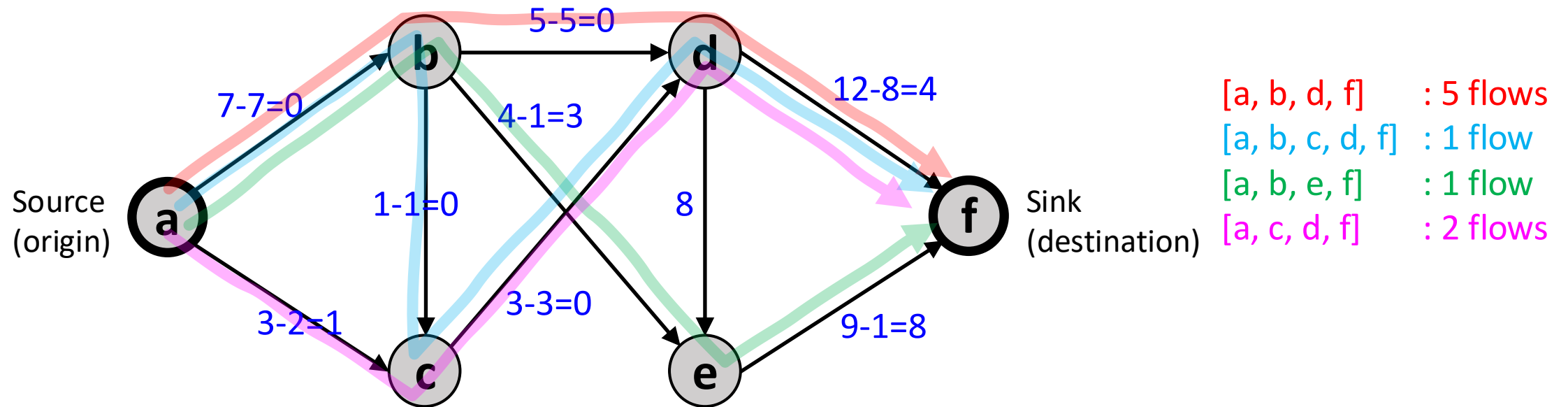
Maximum Flow Problem

- ▶ “With an infinite input source, how much “flow” can we push through the network (from a **source** (origin) to a **sink** (destination) given that each link has a certain **capacity**?”



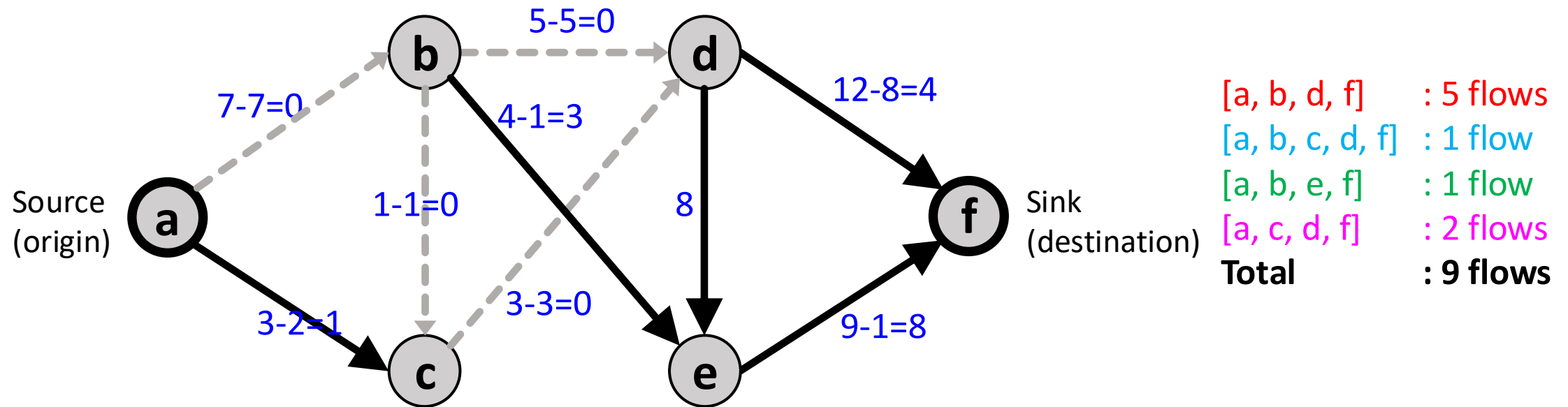
Maximum Flow Problem

- “With an infinite input source, how much “flow” can we push through the network (from a **source** (origin) to a **sink** (destination) given that each link has a certain **capacity**?”



Maximum Flow Problem

► Is this **the maximum flow** for the network?



Maximum Flow Problem

► Real-world Examples in Transportation Networks

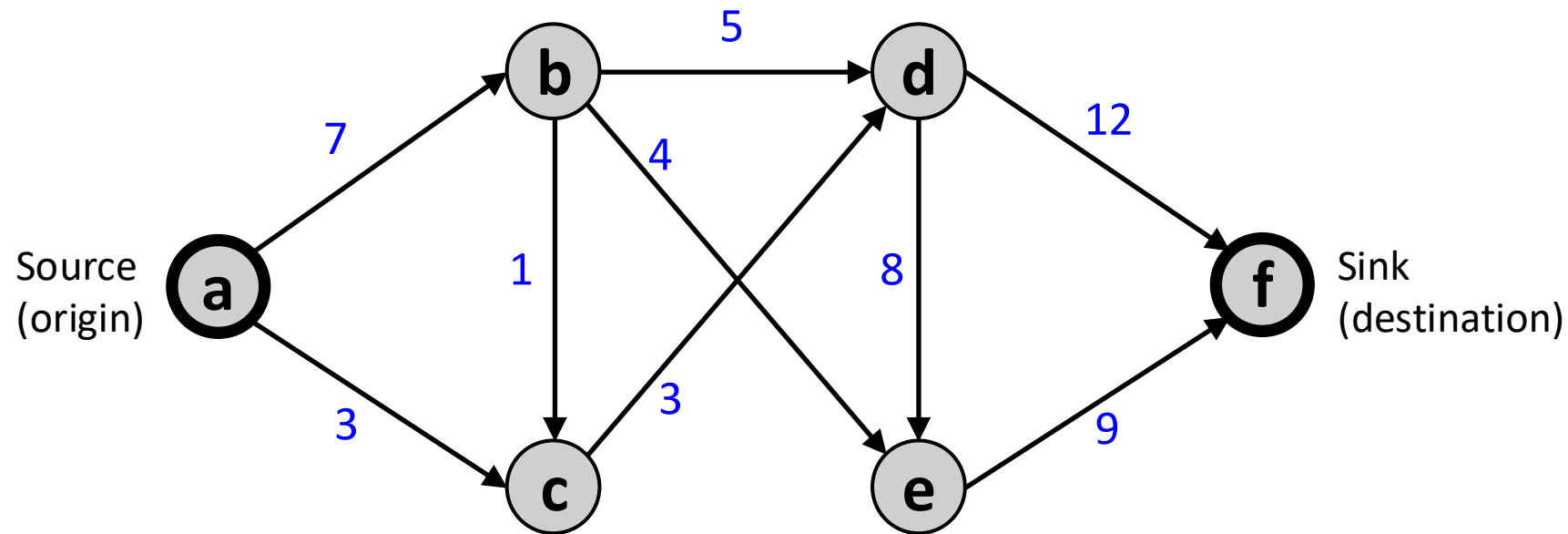
- **Traffic Flow Optimization:** Find the highest possible number of vehicles that can move from one part of a city to another without congestion
- **Public Transport Planning:** Determine the maximum number of passengers that can be transported through a metro system given train capacities and station constraints.
- **Highway and Road Network Design:** Identify bottlenecks and prioritise road expansions
- **Emergency Evacuation Planning:** During a natural disaster (e.g., hurricane), identify optimal escape routes by maximising throughput on highways

► Max-Flow Algorithms

- e.g., Ford–Fulkerson algorithm, Edmonds–Karp algorithm
- The algorithm starts with an initial flow of zero, and iteratively finds a path from the source to the sink that has available capacity, and then increases the flow along that path by the maximum amount possible. This process continues until no more augmenting paths can be found.

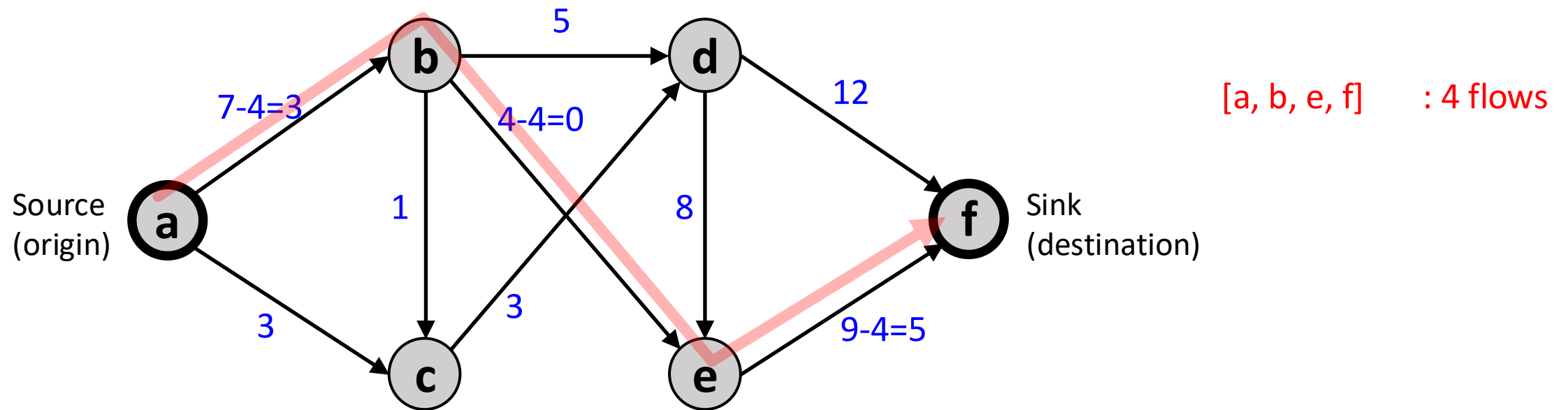
Exercise

- Try to find another path combination that utilises more capacity in the network and sends more flows from **a** to **f**.



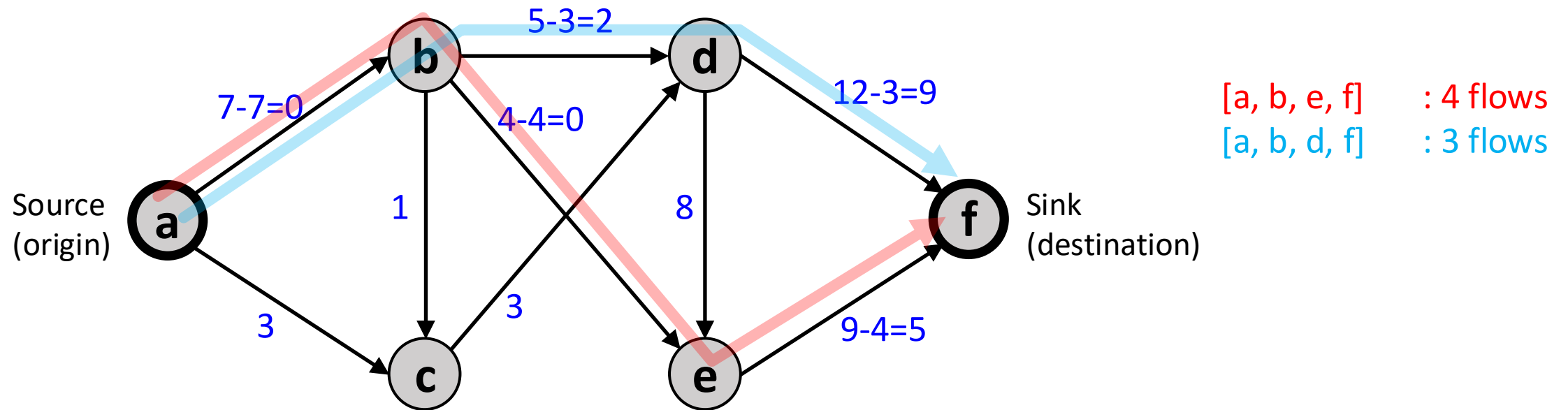
Exercise

- Try to find another path combination that utilises more capacity in the network and sends more flows from **a** to **f**.



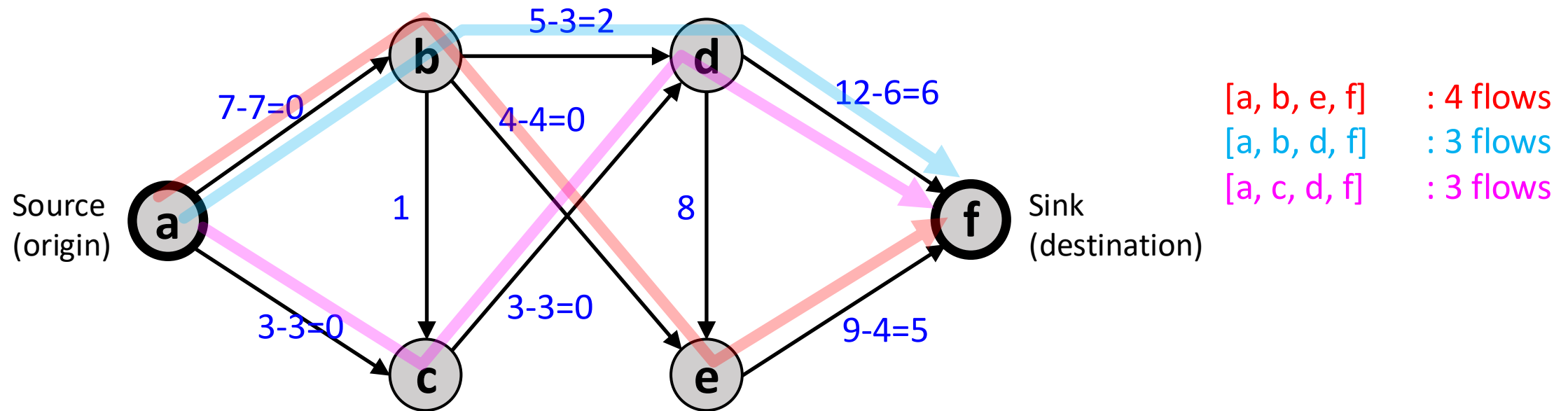
Exercise

- Try to find another path combination that utilises more capacity in the network and sends more flows from **a** to **f**.



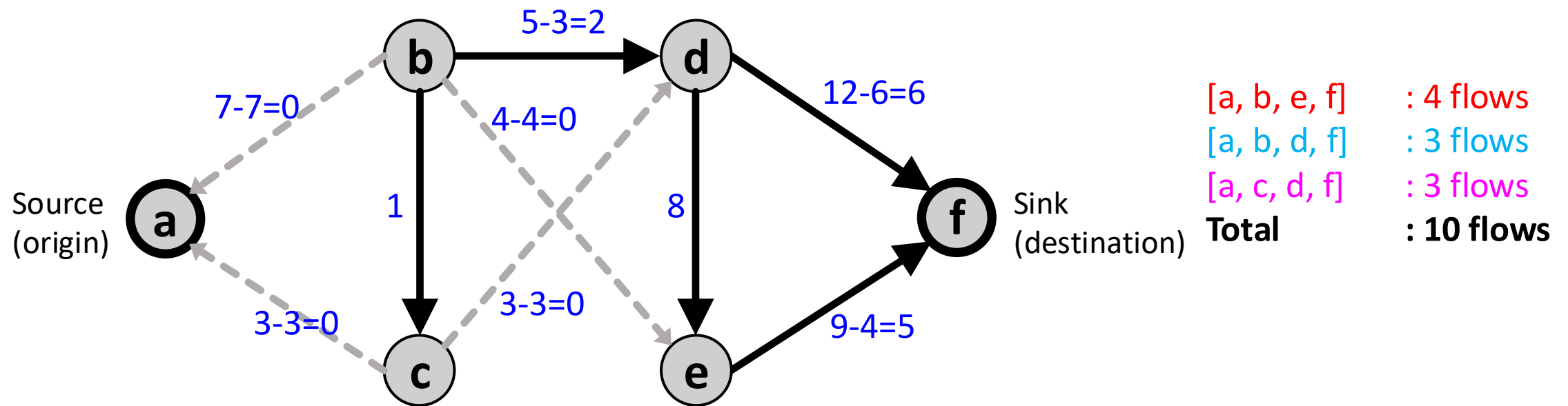
Exercise

- Try to find another path combination that utilises more capacity in the network and sends more flows from **a** to **f**.



Exercise

- This is **the maximum flow** of the network, which is 10.



Thank you